generate the master guide of lecture 1

CS50P: Week 1 / Lecture 1 Master Guide (Conditionals & Control Flow)

1. Comparison & Relational Operators

Branching logic relies on Boolean evaluation (`True` or `False`). Relational operators compare expressions without mutating the operands:

Operator / Syntax	Category	Evaluation Rules & Behavioral Mechanics	Lecture / P Context
`==`	Comparison	Strict value equality; distinguishes evaluation from variable assignment (`=`).	Avoid using `case` patt
`!=`	Comparison	Inequality check; returns `True` if values differ.	Odd parity (`% 2 != 0`
`<` , `<=`	Comparison	Less than / Less than or equal to.	Lower boun (`score <`
`>` , `>=`	Comparison	Greater than / Greater than or equal to.	Upper boun (`score >=`
Chained Comparisons	Mathematical	Python natively evaluates `a <= x <= b` in unified mathematical order.	`7.0 <= t 8.0` in `me`
`%`	Modulo	Computes division remainder; used to test parity and cyclical logic.	Even vs. od `21_parit`

2. Boolean Logic & Short-Circuit Evaluation

Logical conditions connect or invert Boolean expressions using English keywords rather than symbolic operators (`&&` , `||` , `!`):

- `and` **(Conjunction):** Evaluates to `True` only if **both** operands are `True` .
  - *Short-Circuit Behavior:* If the left operand evaluates to `False` , Python immediately halts evaluation and never inspects the right operand.
- `or` **(Disjunction):** Evaluates to `True` if **at least one** operand is `True` .
  - *Short-Circuit Behavior:* If the left operand evaluates to `True` , Python skips evaluating the right operand entirely.
- `not` **(Unary Inversion):** Inverts truth value (`not True` → `False`).
  - Practical usage: Guarding file parsing (`if not "." in file:`).

3. Control Flow Architecture

Independent `if` Blocks (Redundant Check Flaw)

Using sequential `if` statements forces the CPU to evaluate every condition individually, even if a preceding branch already evaluated to `True` :

Python

```python
# Inefficient: Evaluates all 3 expressions every single run
if x < y:
    print("x is smaller than y")
if x > y:
    print("x is bigger than y")
if x == y:
    print("x is equal to y")
```

Mutually Exclusive Architecture (`elif` / `else`)

Grouping checks into `elif` and `else` guarantees that at most one branch executes. Once a condition evaluates to `True` , all subsequent branches are skipped:

Python

```python
# Optimal: Mutually exclusive, stops at the first True condition
if x < y:
    print("x is smaller than y")
elif x > y:
    print("x is bigger than y")
else:
    print("x is equal to y")
```

4. Lecture 1 Code Experiments & Evolution (`16_if.py` to `26_match2.py`)

Evolution of Parity & Boolean Returns

Lecture 1 showcases how conditional code matures from nested branches to idiomatic single-line expressions:

- **Explicit Branches (`22_bool.py`):**

Python

```python
def is_even(n):
    if n % 2 == 0:
        return True
    else:
        return False
```

- **Ternary Operator (`23_easy_return.py`):**

Python

```python
def is_even(n):
    return True if n % 2 == 0 else False
```

- **Idiomatic Pythonic Boolean (`24_easy_return2.py`):**

Python

```python
def is_even(n):
    return n % 2 == 0
```

Key Insight: The comparison `n % 2 == 0` itself evaluates directly to a native Boolean (`True` or `False`), making conditional branching redundant.

Structural Pattern Matching (`25_match.py` & `26_match2.py`)

Introduced in Python 3.10 as an alternative to long `if` / `elif` ladders:

Python

```python
name = input("What's your name? ")

match name:
    case "Harry" | "Hermione" | "Ron":  # Pipe (|) operator groups multiple matching
        print("Gryffindor")
    case "Draco":
        print("Slytherin")
    case _:                             # Wildcard pattern: matches any unhandled inp
        print("Who?")
```

5. String Slicing & Boundary Safety

String slicing follows the `sequence[start:stop]` syntax. Slicing never raises an `IndexError` on out-of-range boundaries—it safely returns whatever characters are present within the range:

Slice Expression	Slicing Behavior	Real-World Application
`name[-4:]`	Returns trailing 4 characters up to the end	Inspecting extensions (`.gif`, `.png`, `.pdf`)
`name[:5]`	Extracts the first 5 characters (indexes 0 to 4)	Safe prefix matching (`greeting[:5] == "hello"`)
`name[4:]`	Skips the first 4 characters, returns the rest	Stripping known prefixes or protocol markers

Slice Expression	Slicing Behavior	Real-World Application
`name[:-4]`	Trims the last 4 characters from the string	Removing a file extension from a base filename

6. Runtime Pitfalls, Common Errors & Solutions

Error / Failure	Code Pattern Causing Error	Root Cause & Resolution
`SyntaxError: invalid syntax`	`case y == "+":`	`case` patterns evaluate identity and value matches directly. Do not write `==` inside `case` lines; write `case "+":`.
`SyntaxError: cannot assign`	`if file[-5:] = "jpeg":`	A single `=` denotes variable assignment. Comparison requires double equals `==`.
`IndexError` / `ValueError`	`prefix, ext = file.rsplit(".", 1)`	Direct unpacking crashes on dotless files (e.g., `"testfile"`). Guard first with `if "." in file:`.
`TypeError: unsupported operand`	`result = x + y` (where `y = "+"`)	Performing math on string operators instead of operand variables (`z`). Always verify variable targets.
`UnboundLocalError`	Variable defined only inside non-exhaustive `if` / `case` branches	The variable is called in `print()` after execution skipped the branch assigning it. Initialize default values before branching.
`check50` **Mismatches**	`input("Enter expression: ")`	Automated tests require exact prompt string matches. Omit or tailor prompt strings to problem specifications.

7. Problem Set Technical Post-Mortem

- **Deep Thought (`deep.py`):**
  - *Objective:* Recognize the answer 42 in numeric and string form.

    - *Key Mechanics:* Chain `s.strip().lower()` immediately on input. Use `match-case` with `|` or chained `or` checks (`"42" | "forty-two" | "forty two"`). Output exact title-cased strings (`"Yes"` / `"No"`).
- **Home Federal Savings Bank (`bank.py`):**
    - *Objective:* Payout $0, $20, or $100 depending on greeting.
    - *Key Mechanics:* Check precedence matters: evaluate `"hello"` before `"h"`. Checking `"h"` first erroneously routes `"hello"` to the $20 tier. Use `.startswith()` or safe slicing `s[:5]`.
- **File Extensions (`extensions.py`):**
    - *Objective:* Map file suffix to its correct MIME type.
    - *Key Mechanics:* Check `if "." in file:` to prevent unpacking errors on dotless files. Strip whitespace, convert to `.lower()`, and handle multi-dot filenames using `file.rsplit(".", 1)[1]`. Default unknown suffixes to `"application/octet-stream"`.
- **Math Interpreter (`interpreter.py`):**
    - *Objective:* Parse string arithmetic (`x y z`) and calculate float results.
    - *Key Mechanics:* Unpack with `x, y, z = expression.split(" ")`. Cast `x` and `z` to `float()`. Format the evaluated result strictly to 1 decimal place using `f"{result:.1f}"`.
- **Meal Time (`meal.py`):**
    - *Objective:* Determine mealtime windows from 24-hour time strings.
    - *Key Mechanics:* Decouple logic cleanly. Dedicated helper `convert(time_str)` parses `hours` and `minutes`, returning `hours + (minutes / 60.0)`. The `main()` driver handles input and runs chained comparisons (`7.0 <= time <= 8.0`).